


**DEPARTMENT OF INFORMATION TECHNOLOGY**

**24ITT32 - PROGRAMMING IN JAVA**

# INDIVIDUAL PROJECT ON ACADEMIC MANAGEMENT SYSTEM

**P R O J E C T R E P O R T**

Submitted by

TRISHNA A K

25ITR164

# JAVA ASSIGNMENT

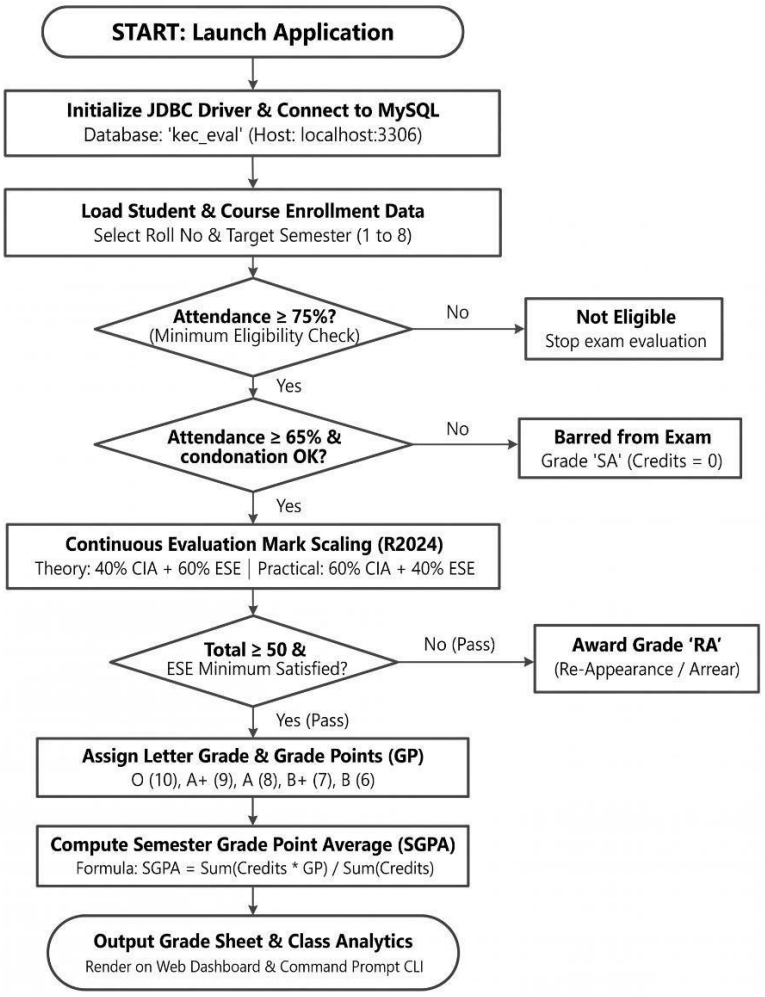
## AIM:

To develop a Student Academic and Continuous Evaluation System using Java, JDBC, and MySQL to manage student details, course information, attendance, assessment marks, semester mark sheets, and department performance reports.

## OBJECTIVE:

Design and develop a database-driven Java application using JDBC to manage student records, perform incremental marks entry (Continuous Internal Assessments and End Semester Examinations), enforce attendance eligibility rules, and generate formal academic performance reports according to Kongu Engineering College (Autonomous) - Regulation 2024. System Requirements & Specifications.

## FLOW CHART:



## PROCEDURE:

- Load the MySQL Connector/J driver to enable communication between the Java application and the MySQL database.

- Establish a connection to the kec_academic database running on the local MySQL server using DriverManager.getConnection().

- Store and retrieve student details, including roll number, name, department, and semester, along with course information such as course code, title, credits, and course type.

- Enroll students in semester courses, record the number of classes held and attended, and calculate the attendance percentage.

- Verify examination eligibility based on attendance percentage. Students with 75% or above are eligible, those with 65%–74% require medical condonation verification, and those below 65% are marked as Shortage of Attendance (SA).

- Record CIA-1, CIA-2, and assignment marks, and calculate the scaled internal assessment marks according to the specified theory and practical course weightages.

- Record end-semester examination marks out of 100, verify the minimum passing requirements, and scale the marks according to the applicable theory and practical .

- Calculate the total marks by combining scaled CIA and ESE marks. Assign letter grades and grade points according to the specified grading scale, marking students as RA if they fail the examination or score below the required passing mark.

- Generate student grade sheets, attendance shortage reports, and department-wise pass percentage reports. Display the results through the interactive web dashboard and the command prompt console.

## DATABASE SCHEMA:

### 1.STUDENTS TABLE:

```
mysql> describe students;
+------------------+-------------+------+-----+---------+----------------+
| Field            | Type        | Null | Key | Default | Extra          |
+------------------+-------------+------+-----+---------+----------------+
| student_id       | bigint      | NO   | PRI | NULL    | auto_increment |
| roll_number      | varchar(20) | NO   | UNI | NULL    |                |
| student_name     | varchar(120)| NO   |     | NULL    |                |
| department       | varchar(40) | NO   |     | NULL    |                |
| current_semester | tinyint     | NO   |     | NULL    |                |
+------------------+-------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

### 2. COURSES TABLE:

```
mysql> describe courses;
+--------------+-------------+------+-----+---------+----------------+
| Field        | Type        | Null | Key | Default | Extra          |
+--------------+-------------+------+-----+---------+----------------+
| course_id    | bigint      | NO   | PRI | NULL    | auto_increment |
| course_code  | varchar(20) | NO   | UNI | NULL    |                |
| course_title | varchar(160)| NO   |     | NULL    |                |
| credits      | decimal(4,1)| NO   |     | NULL    |                |
| course_type  | varchar(12) | NO   |     | NULL    |                |
+--------------+-------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

### 3. ENROLLMENTS TABLE:

```
mysql> describe enrollments;
+------------------+---------+------+-----+---------+----------------+
| Field            | Type    | Null | Key | Default | Extra          |
+------------------+---------+------+-----+---------+----------------+
| enrollment_id    | bigint  | NO   | PRI | NULL    | auto_increment |
| student_id       | bigint  | NO   | MUL | NULL    |                |
| course_id        | bigint  | NO   | MUL | NULL    |                |
| semester         | tinyint | NO   | MUL | NULL    |                |
| classes_held     | int     | NO   |     | 0       |                |
| classes_attended | int     | NO   |     | 0       |                |
+------------------+---------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

### 4. ASSESSMENT MARKS TABLE:

```
mysql> describe assessment_marks;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| enrollment_id | bigint       | NO   | PRI | NULL    |       |
| cia_test_1    | decimal(5,2) | YES  |     | NULL    |       |
| cia_test_2    | decimal(5,2) | YES  |     | NULL    |       |
| phase_2_score | decimal(5,2) | YES  |     | NULL    |       |
| ese_mark      | decimal(5,2) | YES  |     | NULL    |       |
| ese_absent    | tinyint(1)   | NO   |     | 0       |       |
+---------------+--------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

## SOURCE CODE:

### MAINAPP.JAVA:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

public final class MainApp {
    private final Scanner scanner = new Scanner(System.in);
    private final StudentDAO studentDAO = new StudentDAO();
    private final AcademicDAO academicDAO = new AcademicDAO();

    public static void main(String[] args) {
        try (var ignored = DBConnection.getConnection()) {
            new MainApp().run();
        } catch (SQLException exception) {
            System.err.println("Database error: " + exception.getMessage());
            System.err.println("Check MySQL, kec_schema.sql, and KEC_DB_URL /
KEC_DB_USER / KEC_DB_PASSWORD.");
        }
    }

    private void run() {
        while (true) {
            System.out.println("""

                === KEC R2024 Student Academic & Continuous Evaluation System ===
```

```
                1. Student Management
                2. Course Management
                3. Enrollment / Attendance
                4. Enter Marks
                5. Student Semester Mark Sheet
                6. Attendance Defaulters (<65%)
                7. Department Summary
                0. Exit
                """);
            int choice = readInt("Select: ");
            if (choice == 0) {
                System.out.println("Exiting Academic Management System.");
                return;
            }
            try {
                switch (choice) {
                    case 1 -> studentMenu();
                    case 2 -> courseMenu();
                    case 3 -> enrollmentAttendanceMenu();
                    case 4 -> marksMenu();
                    case 5 -> printMarkSheet();
                    case 6 -> printAttendanceDefaulters();
                    case 7 -> printClassSummary();
                    default -> System.out.println("Choose a menu option from 0 to 7.");
                }
            } catch (SQLException exception) {
                System.err.println("Database operation failed: " + exception.getMessage());
            } catch (IllegalArgumentException exception) {
                System.err.println("Input error: " + exception.getMessage());
            }
        }
    }

    private void studentMenu() throws SQLException {
        System.out.println("\n--- Student Management ---\n1. Register student\n2. List
students\n0. Back to main menu");
        switch (readInt("Select: ")) {
            case 1 -> registerStudent();
            case 2 -> listStudents();
            case 0 -> { }
            default -> System.out.println("Invalid student-menu option.");
        }
    }

    private void courseMenu() throws SQLException {
        System.out.println("\n--- Course Management ---\n1. Add course\n2. List
courses\n0. Back to main menu");
        switch (readInt("Select: ")) {
            case 1 -> addCourse();
            case 2 -> listCourses();
```

```
            case 0 -> { }
            default -> System.out.println("Invalid course-menu option.");
        }
    }

    private void enrollmentAttendanceMenu() throws SQLException {
        System.out.println("\n--- Enrollment / Attendance ---\n1. Enroll student in
course\n2. Update attendance totals\n0. Back to main menu");
        switch (readInt("Select: ")) {
            case 1 -> enroll();
            case 2 -> updateAttendance();
            case 0 -> { }
            default -> System.out.println("Invalid enrollment/attendance option.");
        }
    }

    private void marksMenu() throws SQLException {
        System.out.println("\n--- Enter Marks ---\n1. CIA Test 1 / CIA Test 2\n2. Phase
2 / assignment score\n3. ESE mark / mark absent\n0. Back to main menu");
        switch (readInt("Select: ")) {
            case 1 -> enterCiaMarks();
            case 2 -> enterPhase2();
            case 3 -> enterEse();
            case 0 -> { }
            default -> System.out.println("Invalid marks-menu option.");
        }
    }

    private void registerStudent() throws SQLException {
        studentDAO.addStudent(new StudentDAO.Student(
                readText("Roll number: "),
                readText("Name: "),
                readText("Department: "),
                readInt("Current semester (1-8): ")));
        System.out.println("Student registered.");
    }

    private void addCourse() throws SQLException {
        String code = readText("Course code: ");
        String title = readText("Course title: ");
        BigDecimal credits = readDecimal("Credits: ");
        EvaluationEngine.CourseType type =
EvaluationEngine.CourseType.from(readText("Type
(THEORY/PRACTICAL/EMBEDDED): "));
        academicDAO.addCourse(new AcademicDAO.Course(code, title, credits,
type));
        System.out.println("Course added.");
    }

    private void enroll() throws SQLException {
```

```
        academicDAO.enroll(readText("Roll number: "), readText("Course code: "),
readInt("Semester: "));
        System.out.println("Enrollment created.");
    }

    private void updateAttendance() throws SQLException {
        String roll = readText("Roll number: ");
        String course = readText("Course code: ");
        int semester = readInt("Semester: ");
        int held = readInt("Total classes held: ");
        int attended = readInt("Total classes attended: ");
        academicDAO.updateAttendance(roll, course, semester, held, attended);
        System.out.println("Attendance totals updated.");
    }

    private void enterCiaMarks() throws SQLException {
        String roll = readText("Roll number: ");
        String course = readText("Course code: ");
        int semester = readInt("Semester: ");
        BigDecimal cia1 = readMark("CIA Test 1 (0-100): ");
        academicDAO.recordScore(roll, course, semester, "CIA1", cia1, false);
        BigDecimal cia2 = readMark("CIA Test 2 (0-100): ");
        academicDAO.recordScore(roll, course, semester, "CIA2", cia2, false);
        System.out.println("CIA marks recorded.");
    }

    private void enterPhase2() throws SQLException {
        academicDAO.recordScore(
                readText("Roll number: "),
                readText("Course code: "),
                readInt("Semester: "),
                "PHASE2",
                readMark("Assignments / quizzes / lab evaluation score (0-100): "),
                false);
        System.out.println("Phase 2 score recorded.");
    }

    private void enterEse() throws SQLException {
        String roll = readText("Roll number: ");
        String course = readText("Course code: ");
        int semester = readInt("Semester: ");
        String absentInput = readText("Absent in ESE? (y/n): ");
        if ("y".equalsIgnoreCase(absentInput)) {
            academicDAO.recordScore(roll, course, semester, "ESE", null, true);
            System.out.println("ESE absence recorded.");
        } else if ("n".equalsIgnoreCase(absentInput)) {
            academicDAO.recordScore(roll, course, semester, "ESE", readMark("ESE
mark (0-100): "), false);
            System.out.println("ESE mark recorded.");
        } else {
```

```
            throw new IllegalArgumentException("Enter y or n.");
        }
    }

    private void printMarkSheet() throws SQLException {
        String roll = readText("Roll number: ");
        int semester = readInt("Semester: ");
        List<AcademicDAO.CourseRow> rows =
academicDAO.getSemesterRows(semester, roll);
        if (rows.isEmpty()) {
            System.out.println("No enrollment found for that student and semester.");
            return;
        }
        System.out.printf("%nKEC R2024 STUDENT SEMESTER MARK
SHEET%n%s | %s | %s | Semester %d%n",
                rows.get(0).rollNumber(), rows.get(0).studentName(),
rows.get(0).department(), semester);
        System.out.printf("%-10s %-18s %6s %6s %6s %8s %8s %7s %8s %8s %-6s
%-7s %s%n",
                "Course", "Title", "CIA1", "CIA2", "Phase2", "Internal", "CIA Wt.",
                "ESE Raw", "ESE Wt.", "Total", "Grade", "Status", "Attendance");

        List<EvaluationEngine.SemesterCourse> semesterCourses = new ArrayList<>();
        boolean complete = true;
        for (AcademicDAO.CourseRow row : rows) {
            EvaluationEngine.Result result = row.result();
            semesterCourses.add(new EvaluationEngine.SemesterCourse(row.credits(),
result));
            complete &= result.complete();
            System.out.printf("%-10s %-18s %6s %6s %6s %8s %8s %7s %8s %8s %-6s
%-7s %s%n",
                    row.courseCode(), truncate(row.courseTitle(), 18),
                    format(row.cia1()), format(row.cia2()), format(row.phase2()),
                    format(result.internalScore()), format(result.internalWeighted()),
                    format(row.eseMark()), format(result.eseWeighted()),
format(result.total()),
                    result.grade(), result.status(), format(result.attendancePercent()) + "%");
        }

        BigDecimal sgpa = complete ?
EvaluationEngine.semesterGpa(semesterCourses) : null;
        System.out.println("SGPA: " + format(sgpa));
        System.out.println("Equivalent percentage: " + (sgpa == null ? "PENDING"
                : sgpa.multiply(BigDecimal.TEN).setScale(2, RoundingMode.HALF_UP)
+ "%"));
        System.out.println("CIA Wt. and ESE Wt. are weighted contributions to the total
out of 100.");
    }

    private void printAttendanceDefaulters() throws SQLException {
```

```
        List<AcademicDAO.CourseRow> rows =
academicDAO.getAttendanceDefaulters();
        System.out.println("\nATTENDANCE DEFAULTERS (<65%)");
        if (rows.isEmpty()) {
            System.out.println("No attendance defaulters found.");
            return;
        }
        System.out.printf("%-12s %-22s %-10s %-12s %-8s %-12s %-8s%n",
                "Roll", "Name", "Department", "Course", "Semester", "Attendance",
"Grade");
        for (AcademicDAO.CourseRow row : rows) {
            EvaluationEngine.Result result = row.result();
            System.out.printf("%-12s %-22s %-10s %-12s %-8d %-12s %-8s%n",
                    row.rollNumber(), truncate(row.studentName(), 22), row.department(),
                    row.courseCode(), row.semester(), format(result.attendancePercent()) +
"%", result.grade());
        }
    }

    private void printClassSummary() throws SQLException {
        int semester = readInt("Semester: ");
        List<AcademicDAO.CourseRow> rows =
academicDAO.getSemesterRows(semester, null);
        Map<String, Map<String, List<AcademicDAO.CourseRow>>> departments =
new LinkedHashMap<>();
        for (AcademicDAO.CourseRow row : rows) {
            departments.computeIfAbsent(row.department(), ignored -> new
LinkedHashMap<>())
                    .computeIfAbsent(row.rollNumber(), ignored -> new
ArrayList<>()).add(row);
        }
        System.out.printf("%nDEPARTMENT PERFORMANCE SUMMARY -
SEMESTER %d%n", semester);
        System.out.printf("%-30s %8s %8s %10s %12s%n", "Department", "Pass",
"Fail", "Pending", "Avg SGPA");
        for (Map.Entry<String, Map<String, List<AcademicDAO.CourseRow>>>
department : departments.entrySet()) {
            int pass = 0;
            int fail = 0;
            int pending = 0;
            BigDecimal sgpaSum = BigDecimal.ZERO;
            int gpaCount = 0;
            for (List<AcademicDAO.CourseRow> studentRows :
department.getValue().values()) {
                List<EvaluationEngine.SemesterCourse> courses = new ArrayList<>();
                boolean allComplete = true;
                boolean hasFailure = false;
                for (AcademicDAO.CourseRow row : studentRows) {
                    EvaluationEngine.Result result = row.result();
                    courses.add(new EvaluationEngine.SemesterCourse(row.credits(),
```

```
result));
                allComplete &= result.complete();
                hasFailure |= "RA".equals(result.grade()) || "SA".equals(result.grade())
                        || "AB".equals(result.grade());
            }
            if (hasFailure) {
                fail++;
            } else if (!allComplete) {
                pending++;
            } else {
                pass++;
            }
            if (allComplete) {
                BigDecimal gpa = EvaluationEngine.semesterGpa(courses);
                if (gpa != null) {
                    sgpaSum = sgpaSum.add(gpa);
                    gpaCount++;
                }
            }
        }
        BigDecimal average = gpaCount == 0 ? null
                : sgpaSum.divide(BigDecimal.valueOf(gpaCount), 2,
RoundingMode.HALF_UP);
        System.out.printf("%-30s %8d %8d %10d %12s%n",
                department.getKey(), pass, fail, pending, format(average));
      }
      if (departments.isEmpty()) {
        System.out.println("No enrollments found for that semester.");
      }
      System.out.println("Students with any RA/SA/AB are counted as fail;
incomplete results as pending.");
   }
   private void listStudents() throws SQLException {
      System.out.println("\nSTUDENTS");
      List<StudentDAO.Student> students = studentDAO.listStudents();
      if (students.isEmpty()) System.out.println("No students found.");
      for (StudentDAO.Student student : students) {
        System.out.printf("%-12s %-24s %-24s semester %d%n",
                student.rollNumber(), student.name(), student.department(),
student.currentSemester());
      }
   }

   private void listCourses() throws SQLException {
      System.out.println("\nCOURSES");
      List<AcademicDAO.Course> courses = academicDAO.listCourses();
      if (courses.isEmpty()) System.out.println("No courses found.");
      for (AcademicDAO.Course course : courses) {
        System.out.printf("%-12s %-32s %s credits %s%n",
                course.code(), truncate(course.title(), 32), course.type(), course.credits());
```

```
        }
    }

    private int readInt(String prompt) {
        try {
            return Integer.parseInt(readText(prompt));
        } catch (NumberFormatException exception) {
            throw new IllegalArgumentException("Enter a whole number.");
        }
    }

    private BigDecimal readDecimal(String prompt) {
        try {
            return new BigDecimal(readText(prompt));
        } catch (NumberFormatException exception) {
            throw new IllegalArgumentException("Enter a valid decimal number.");
        }
    }

    private BigDecimal readMark(String prompt) {
        BigDecimal mark = readDecimal(prompt);
        if (mark.compareTo(BigDecimal.ZERO) < 0 ||
mark.compareTo(BigDecimal.valueOf(100)) > 0) {
            throw new IllegalArgumentException("Marks must be between 0 and 100.");
        }
        return mark;
    }

    private String readText(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
    }

    private static String format(BigDecimal value) {
        return value == null ? "--" : value.setScale(2,
RoundingMode.HALF_UP).toPlainString();
    }

    private static String truncate(String value, int maxLength) {
        return value.length() <= maxLength ? value : value.substring(0, maxLength - 3)
+ "...";
    }
}
```

## DB CONNECTION.JAVA:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class DBConnection {
   private static final String DEFAULT_URL =
"jdbc:mysql://localhost:3306/kec_academic";

   private DBConnection() {
   }

   public static Connection getConnection() throws SQLException {
      String url = setting("KEC_DB_URL", DEFAULT_URL);
      String user = System.getenv("KEC_DB_USER");
      String password = System.getenv("KEC_DB_PASSWORD");

      if (user == null || user.isBlank()) {
         throw new SQLException("Set the KEC_DB_USER environment variable before
starting the application.");
      }
      if (password == null) {
         throw new SQLException("Set the KEC_DB_PASSWORD environment variable
before starting the application.");
      }

      return DriverManager.getConnection(url, user, password);
   }

   private static String setting(String name, String fallback) {
      String value = System.getenv(name);
      return value == null || value.isBlank() ? fallback : value;
   }
}
```

## OUTPUT SCREENSHOTS:

### MAIN MENU:

```
=== KEC R2024 Student Academic & Continuous Evaluation System ===
1. Student Management
2. Course Management
3. Enrollment / Attendance
4. Enter Marks
5. Student Semester Mark Sheet
6. Attendance Defaulters (<65%)
7. Department Summary
0. Exit
```

## STUDENT & COURSE MANAGEMENT:

### 1.STUDENT REGISTRATION:

```
KEC R2024 Student Academic & Continuous Evaluation System

1. Register student
2. Add course
3. Enroll student in course
4. Update attendance / condonation
5. Enter or update assessment marks
6. Student semester mark sheet
7. Attendance defaulter report
8. Department result summary
0. Exit

Choose an option: 1
Roll number: 25ITR164
Student name: TRISHNA A K
Department: IT
Current semester: 3
Student registered.
```

### 2. COURSE CATALOG:

```
--- Course Management ---
1. Add course
2. List courses
0. Back to main menu
Select: 1
Course code: 24ITC55
Course title: Animation Lab
Credits: 1
Type (THEORY/PRACTICAL/EMBEDDED): PRACTICAL
Course added.
```

### 3. COURSE ENROLLMENT & ATTENDANCE TRACKING:

```
--- Enrollment / Attendance ---
1. Enroll student in course
2. Update attendance totals
0. Back to main menu
Select: 2
Roll number: 24CSR777
Course code: 23ACR66
Semester: 5
Total classes held: 77
Total classes attended: 70
Attendance totals updated.
```

## INCREMENTAL MARKS ENTRY:

### 1.PHASE 1 (MID-SEMESTER):

```
--- Enter Marks ---
1. CIA Test 1 / CIA Test 2
2. Phase 2 / assignment score
3. ESE mark / mark absent
0. Back to main menu
Select: 1
Roll number: 25ALR018
Course code: 23ACR66
Semester: 3
CIA Test 1 (0-100): 88
CIA Test 2 (0-100): 89
CIA marks recorded.
```

### 2.PHASE 2 (END OF SEMESTER - CIA):

```
--- Enter Marks ---
1. CIA Test 1 / CIA Test 2
2. Phase 2 / assignment score
3. ESE mark / mark absent
0. Back to main menu
Select: 2
Roll number: 25ALR018
Course code: 23ACR66
Semester: 3
Assignments / quizzes / lab evaluation score (0-100): 97
Phase 2 score recorded.
```

### 3. PHASE 3 (POST SEMESTER):

```
--- Enter Marks ---
1. CIA Test 1 / CIA Test 2
2. Phase 2 / assignment score
3. ESE mark / mark absent
0. Back to main menu
Select: 3
Roll number: 25ALR018
Course code: 23ACR66
Semester: 3
Absent in ESE? (y/n): n
ESE mark (0-100): 93
ESE mark recorded.
```

## ATTENDANCE & GRADING RULES:

### 1.ATTENDANCE FILTER:

```
1. Register student
2. Add course
3. Enroll student in course
4. Update attendance / condonation
5. Enter or update assessment marks
6. Student semester mark sheet
7. Attendance defaulter report
8. Department result summary
0. Exit

Choose an option: 7

ATTENDANCE DEFAULTERS (below 65%) - Grade SA
Roll        Name                Department Sem     Course      Attendance   Grade
```

# STUDENT SEMESTER MARK SHEET:

```
Select: 5
Roll number: 25ALR018
Semester: 3

KEC R2024 STUDENT SEMESTER MARK SHEET
25ALR018 | Ithazhya S | AI&ML | Semester 3
Course     Title               CIA1   CIA2 Phase2 Internal  CIA Wt. ESE Raw  ESE Wt.    Total Grade  Status  Attendance
18CSC65    System Design         --     --     --       --       --      --       --       -- PENDING ATTENDANCE NOT RECORDED --%
23ACR66    Foundation of AI    88.00  89.00  97.00    91.33    36.53   93.00    55.80    92.33 O      PASS    100.00%
24ITC55    Animation Lab         --     --     --       --       --      --       --       -- PENDING ATTENDANCE NOT RECORDED --%
SGPA: --
Equivalent percentage: PENDING
CIA Wt. and ESE Wt. are weighted contributions to the total out of 100.
```

# DEPARTMENT WISE CLASS PERFORMANCE & RESULT SUMMARY:

```
Select: 7
Semester: 3

DEPARTMENT PERFORMANCE SUMMARY - SEMESTER 3
Department                      Pass     Fail    Pending     Avg SGPA
AI&ML                              0        0          1           --
INFORMATION TECHNOLOGY             1        1          0         9.00
Students with any RA/SA/AB are counted as fail; incomplete results as pending.

Select: 7
Semester: 5

DEPARTMENT PERFORMANCE SUMMARY - SEMESTER 5
Department                        Pass     Fail    Pending     Avg SGPA
CSE                                  0        1          0           --
Students with any RA/SA/AB are counted as fail; incomplete results as pending.
```

# DATABASE:

## STUDENTS DATABASE:

```
mysql> SELECT* FROM students;
+------------+------------+--------------+-----------------------+------------------+
| student_id | roll_number | student_name | department            | current_semester |
+------------+------------+--------------+-----------------------+------------------+
|          2 | 25ITR172    | Vikasini E   | INFORMATION TECHNOLOGY |                3 |
|          3 | 25ITR164    | Trishna A K  | INFORMATION TECHNOLOGY |                3 |
|          5 | 25ALR018    | Ithazhya S   | AI&ML                 |                3 |
|          6 | 24CSR777    | Dhoni M S    | CSE                   |                5 |
|          7 | 19CSR011    | Arul R       | CSE                   |                7 |
|          8 | 25BBA087    | Kanika S     | BBA                   |                3 |
+------------+------------+--------------+-----------------------+------------------+
6 rows in set (0.00 sec)
```

## COURSES DATABASE:

```
mysql> SELECT * FROM courses;
+-----------+-------------+-------------------------------+---------+-------------+
| course_id | course_code | course_title                  | credits | course_type |
+-----------+-------------+-------------------------------+---------+-------------+
|         4 | 24ITT32     | Programming in Java           |     3.0 | EMBEDDED    |
|         5 | 24ITT31     | Information Theory and Coding |     3.0 | THEORY      |
|         6 | 24ITC31     | Full Stack Development        |     3.0 | EMBEDDED    |
|         7 | 24ITC55     | Animation Lab                 |     1.0 | PRACTICAL   |
|         8 | 23ACR66     | Foundation of AI              |     3.0 | THEORY      |
|         9 | 18CSC65     | System Design                 |     2.0 | THEORY      |
|        10 | 22BAC33     | Corporate Law                 |     4.0 | THEORY      |
+-----------+-------------+-------------------------------+---------+-------------+
7 rows in set (0.00 sec)
```

## ENROLLMENTS DATABASE:

```
mysql> SELECT * FROM enrollments;
+---------------+------------+-----------+----------+--------------+------------------+
| enrollment_id | student_id | course_id | semester | classes_held | classes_attended |
+---------------+------------+-----------+----------+--------------+------------------+
|             2 |          2 |         4 |        3 |           48 |               22 |
|             3 |          3 |         5 |        3 |           42 |               40 |
|             4 |          2 |         6 |        3 |           47 |               47 |
|             5 |          2 |         5 |        3 |            0 |                0 |
|             6 |          7 |         9 |        7 |           55 |               47 |
|             7 |          7 |         4 |        7 |           88 |               84 |
|             8 |          5 |         8 |        3 |           77 |               77 |
|             9 |          5 |         9 |        3 |            0 |                0 |
|            10 |          5 |         7 |        3 |            0 |                0 |
|            11 |          6 |        10 |        5 |           66 |               33 |
|            12 |          6 |         4 |        5 |            0 |                0 |
|            13 |          6 |         6 |        5 |           45 |               41 |
|            14 |          6 |         8 |        5 |           77 |               70 |
+---------------+------------+-----------+----------+--------------+------------------+
13 rows in set (0.00 sec)
```

## RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) Regulation 2024 was successfully designed, developed, and tested using Java (JDBC) and MySQL. The system accurately automates the calculation of Continuous Internal Assessments (CIA), End Semester Examinations (ESE), Letter Grades, Attendance Eligibility, and Semester Grade Point Average (SGPA).

# REFERENCES:


**1. Oracle, Java SE Documentation – JDBC Overview and Architecture.**
https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/ .

**2. Oracle, Java SE API – HttpServer.**
https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html .

**3. Oracle, MySQL 8.0 Reference Manual.** https://dev.mysql.com/doc/refman/8.0/en/ .

**4. Oracle, MySQL Connector/J Developer Guide.**
https://dev.mysql.com/doc/connector-j/8.0/en/ .

**5. University Grants Commission, Guidelines for Choice Based Credit System (CBCS).** https://www.ugc.gov.in/pdfnews/8023719_Guidelines-for-CBCS.pdf .

**6. Chart.js, Chart.js Documentation.** https://www.chartjs.org/docs/latest/ .

**7. Tailwind CSS, Tailwind CSS Documentation.** https://tailwindcss.com/docs .

**8. GeeksforGeeks, Establishing JDBC Connection in Java.**
https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/
.

**9. YouTube, Java JDBC with MySQL Tutorial Videos.**
https://www.youtube.com/results?search_query=Java+JDBC+MySQL+tutorial
.

**10. Anthropic, Claude AI Assistant (used for project ideas and guidance).** https://claude.ai/ .


# GITHUB:

| | |
|---|---|
| **LINK:** | **https://github.com/Trishna2503/JDBC-Assignment.git** |